Project Specification

Overview

This project's aim is to develop a simple simulator which can test factory designs before building them in game. Thus, it is purely a command line tool, with no graphical component. The project will take a sample factory configuration file (representing a factory design), run a simulation, and then output a report with statistics based on that simulation.

A factory design consists of a number of machines configured to process an input material, and output another material. Machines may be active, or in cool-down mode. Materials are transported between machines via belts, specifically belt balancers. Belt balancers are systems that take some number of input belts and distribute the throughput from these belts uniformly across some number of output belts (barring any output belts that are blocked). A belt balancer can only handle a single material type.

A simulation does not run in real time. Its goal is to compute the final statistics, so it can run as fast as it can be computed. E.g. because this project is just a simulation, there is no need for us to track the *physical* movement of materials between machines - assuming belts are available they can instead be moved instantly.

To simplify the interactions between machines and belts, the simulation runs on a tick/tock cycle: on each "tick", all the machines update as appropriate, and on each "tock", belt balancers move items between machines. A step consists of a single "tick" followed by a single "tock". Each step represents 1 second of time. See `run` method in the provided Simulator.java class. A simulation will do a specified number of warmup steps, followed by a specified number of statistic collection steps. The Main.java program parses a configuration file (see below), runs a simulation, generates a report, and prints that report.

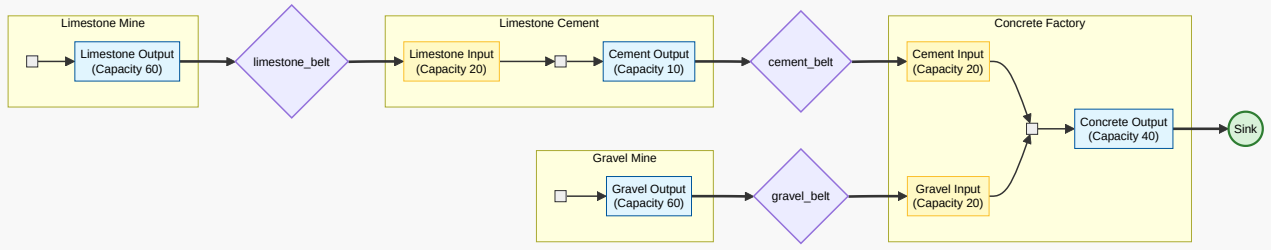
Configuration File

Below is an example configuration file:

```
[config]
warmup_duration: 10000  # Inline comments are supported
statistics_duration: 600

[machine]
name: Limestone Mine
output: Limestone, 60, limestone_belt
cooldown: 1

[machine]
name: Gravel Mine
output: Gravel, 60, gravel_belt
cooldown: 1

[machine]
name: Limestone Cement
input: Limestone, 20, limestone_belt
```

```
output: Cement, 10, cement_belt
cooldown: 1

[machine]
name: Concrete Factory
input: Cement, 20, cement_belt
input: Gravel, 20, gravel_belt
output: Concrete, 40, sink
cooldown: 6
```

This is a visualisation of the factory it represents!



This file is parsed by the config package provided, where the ConfigParser class produces a SimulatorConfig object that represents the data in this file. You will not need to use the ConfigParser class directly, but you should get familiar with the PortConfig, and MachineConfig classes that represent the data given in a config file.

Simulator Requirements

The config file specifies a warm-up duration and statistics_duration. This is used by the Simulator.java file to first run the specified number of warm-up steps (to get the factory into a realistic state), then reset. This has been implemented for you.

Machine-Requirements

Each machine is specified with the following:

- `name`: A human-readable string name for the machine to identify it in statistics, etc.
- `input`: Repeated field. Each input is specified as `item, amount, belt_name`, where `item` is the name of the item, `amount` is the amount of the item required per activation, and `belt_name` is the name of the belt that the item is coming from.
- `output`: Repeated field. Each output is specified as `item, amount, belt_name`, where `item` is the name of the item, `amount` is the amount of the item produced per activation, and `belt_name` is the name of the belt that the item is going to.
- `cooldown`: The number of seconds to wait before the machine can activate again.

A machine has a small amount of internal storage, enough to store one batch of its input and output materials. E.g. A ConcreteFactory can store up to 20 units of Cement input, 20 units of Gravel input, and 40 units of concrete output. You may assume that there is at most one input/output port per item type.

On each tick, a machine will activate if:

- it has all its required input materials
- its output storage (across all ports) is empty
- it is not in a "cooldown" state.

After activation, on the next tick the cool down period will start, and the machine may not activate again until it has elapsed. If a machine has a cooldown period of 4, it means it can activate at most once every 5 ticks (4 cool down ticks + 1 activation tick) to be 100% utilized. A machine is considered blocked if it *is not in a cool down state* but cannot activate because:

- It does not have its required input materials OR
- Its output storage contains items.

A "mine" or other resource source can be implemented as a machine with no inputs (it produces items from nothing). It therefore requires no inputs to activate.

Belt-Balancer Requirements

A belt balancer is a system that takes some number of items from machine output ports and distributes the throughput from these uniformly across some number of (non-blocked) machine input ports. In the config format, belt balancers are simply specified by name as part of the machine config (input/output configuration).

Belt balancers must meet the following requirements:

- Any belt that is named in the config file exists in the factory.
- Belt names are unique, so when two machines name the same belt, they are both connected to it.
- A belt balancer can only carry one type of item (except sink, see below).
- Therefore, if *two machine ports with different item types* try to connect to the same belt (not checked during config parsing!!!) this should throw a BeltValidationException. (Corrected 24/4/26)
- Belt balancers have no cooldown or limit on how fast they can transfer items.

On each tock, a belt balancer pulls materials from machine output ports it is connected to (skipping any that are empty), and distributes it to available machine input ports (skipping those that are full). The maximum number of items a belt can transfer during a tock is determined by the supply (sum of capacity of all connected machine output ports) and demand (sum of remaining capacity of all connected machine input ports). As soon as either value is reached, the belt can no longer transfer items.

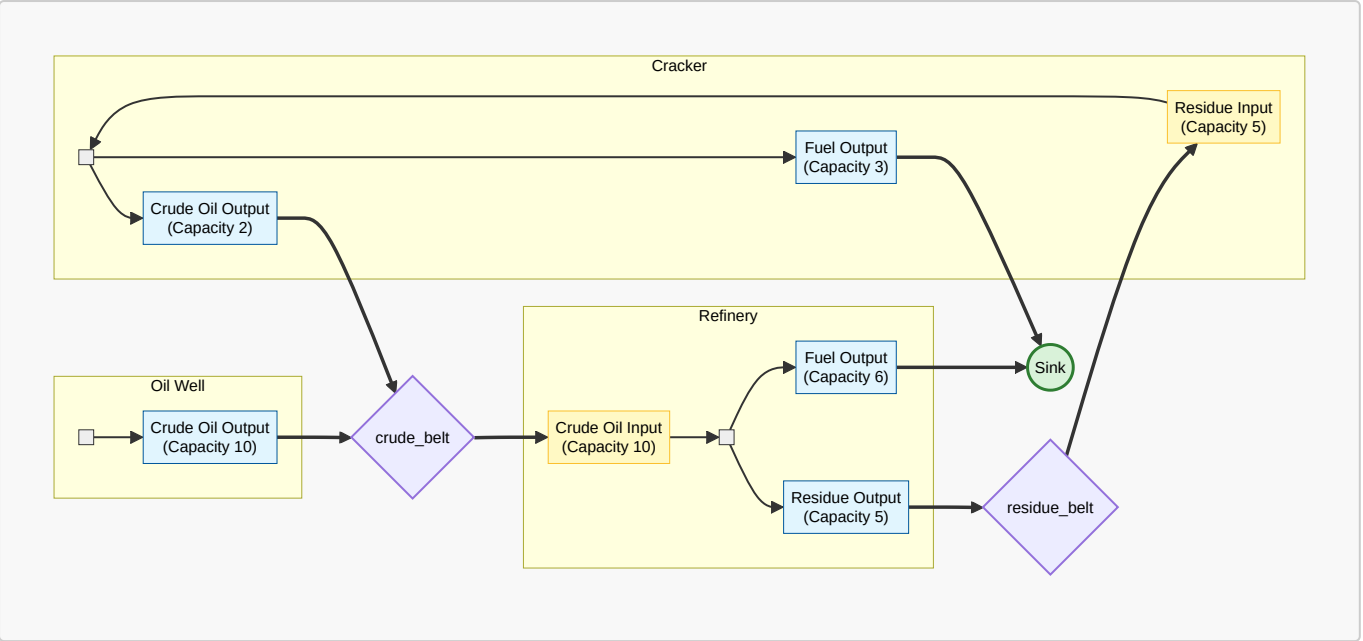
To ensure materials a distributed fairly, it does this in a round-robin fashion, i.e. output/input ports take turns being pulled/pushed from. For each item that can be transferred:

- A single item will be pulled from the output port whose current "turn" it is
- The item will be pushed to the input port whose current "turn" it is.

The output/input port with the current "turn" should be preserved between tocks. e.g. If a tock pushes its last item to Input Port N, on the next tock it should first try to push to input port N+1. The initial turn should be determined by the order the machines appear in the config file.

Output ports that are empty should be skipped until the next output port with items in the cycle is reached. Similarly, Input ports that are full should be skipped.

For example, consider the oil_refinery config example, represented by the diagram below:



(Corrected 24/4/26): The crude belt is connected to two machine output ports, and one machine input port (Refinery). On the first tock of the system:

- The crude-belt will first try to pull a single item from Oil-well (which appears first in the machine config file, so has the initial turn), which it does successfully, and pushes it to the Refinery.
- It will then try to pull an item from the Cracker, however Cracker hasn't produced any output yet! So the Cracker's turn is skipped.
- So it will instead pull again from Oil-well and push an item to the Refinery.
- And continue on until Oil-well has no remaining output, or the Refinery input is full.

Later in the simulation, when say Cracker contains output, on a tock the system will:

- Figure out which machine output port's turn it is (turns are persisted between tocks). Let's say the last item on the last tock was pulled from the Oil Well. That means this tock starts with the Cracker!
- The crude-belt pulls a single item from the Cracker and pushes that to the Refinery
- then pulls a single item from the Oil Well and pushes it to the Refinery.
- then pulls another item from the Cracker and pushes to the Refinery.
- and so on until either the Refinery inputs are full, or both the Oil Well and Cracker outputs are empty (if one empties first, the belt will then repeatedly pull from the other)

Note that this is a critical part of the simulation: If the throughputs of different parts of the factory are not sufficient, it will create a bottleneck, which will appear in the simulation as a blocked machine with full inputs, and hence its input balancer may be blocked and unable to pull items from its input machines, meaning they will end up blocked as well, and this will cascade through, lowering the utilisation of machines and decreasing throughput.

Sink Requirements

The belt name `sink` is reserved for a special sink component that can accept unlimited items of any type. This is used to model the end-point of a factory so we can measure its total output capabilities.

Unlike a belt balancer, a sink:

- Is only connected to machine output ports, and not any machine input ports (items are consumed, not transferred)
- Can accept unlimited items per tick (no backpressure)
- Can accept multiple different item types

During the "tock" phase, the sink pulls all available items from the output storage of any machines that output to it. As it can consume all available items, it does not need to use any "round-robin" process like belts.

If a config file tries to set a sink to connect to a machine input port (Not checked during config parsing!!!), the program should throw a BeltValidationException.

Statistic Requirements

The report (created by the provided Report.java class) will output a number of statistics. These should be calculated over the statistics window (i.e. after the warmup period). Your model will have to be able to produce these statistics, to generate the report. A sample report for the ConcreteFactory model is:

=== Machine Utilisation ===
Limestone Mine: 11.0%
Gravel Mine: 5.7%
Limestone Cement: 33.3%
Concrete Factory: 100.0%

=== Belt Throughput ===
limestone_belt: 400.0 / min (Limestone)
gravel_belt: 200.0 / min (Gravel)
cement_belt: 200.0 / min (Cement)

=== Sink ===
Concrete: 400.0 / min

Machine Statistics

A machine should track the time it is being utilised for (over the duration of the statistics window), presented as a percentage in the above report.

- A machine's utilisation is the percentage of time it is being used during the statistics window (either in cooldown or being activated, i.e. whenever it is not blocked by lack of input or any output port containing items).
- This will be passed to the Report class as a fraction (double between 0.0 and 1.0), which the report then presents as a percentage. E.g. the fraction is: amount of ticks in use / total ticks.

Belt Statistics

A belt balancer should track (over the duration of the statistics window) the number of items it transfers, and present this in the final statistics report as an average number of items per minute.

Average Formula: (count of items transferred / number of tocks) * 60.
(Recalling number of tocks = number of seconds)

Sink Statistics

Similar to a belt, a sink should track (over the duration of the statistics window) the number of items consumed per item type (over the duration of the statistics window), and present this in the final statistics report as an average number of items per minute for each item type.

Average Formula: (count of items consumed / number of tocks) * 60.

The report above only has a single entry under === Sink === as the sink is only connected to one output port. As a sink may collect multiple different types of items, some reports will have more entries here.

Updates

24/04/26: Minor but important corrections to belt-balancer specification. A Belt Balancer can have multiple machine input/output ports connected to it, but they must all have the same item type (otherwise throw BeltValidationException). The Oil refinery example of behaviour has also been corrected to reflect the diagram, and extended to clarify initial turn behaviour.